Qdreon

Gordon Ramsay Restaurant Reservation System

Software Design Document

Arcilla, Reuel Matthew
Belocura, Kisha Faye
Cantago, Dale Ryoko Jose
Castroverde, Mickel Angelo
Durango, Cyrus Alain

Lab Team 1

Date: 04/22/2026

1.0 INTRODUCTION

1.1 Purpose

This software design document describes the architecture and system design of the Gordon Ramsay Restaurant Reservation System, which is a single-tenant platform designed to modernize table booking. The intended audience includes the developer team (Group LT1) responsible for code development, the professors and instructors evaluating the system design, and the client/stakeholders (Group LT3).

1.2 Scope

The Gordon Ramsay Restaurant Reservation System is a web-based, single-tenant application that connects diners with restaurant management for one dedicated restaurant location.

The primary goal of the system is to replace manual reservation logbooks with a scalable digital environment that optimizes daily operations and offers customers a frictionless booking experience.

The product is scoped to include two main modules: the **Customer Portal** and the **Restaurant Management System**.

Customer Benefits:

- An online discovery engine with integrated “view-only” menus.
- Virtual waitlists.
- Automated Email notifications to prevent double bookings.

Restaurant Administrator Benefits

- Robust operations control via a static but real-time interactive floor plan grid.
- Guest CRM tracking.
- Deposit handling capabilities.

1.3 Overview

This Document outlines the design decisions to satisfy the requirements defined in the Software Requirements Specification for the Gordon Ramsey Restaurant Reservation System:

Section 2.0 System Overview

Section 3.0 System Architecture

Section 4.0 Data Design

Section 5.0 Component Design

Section 6.0 Human Interface Design

Section 7.0 Requirements Matrix

1.4 Reference Material

The following documents were used as sources of information and design guidance:

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998.

Gomaa, H. (2011). Software Modelling and Design: UML, Use Cases, Patterns, and Software Architectures (COMET Method). Cambridge University Press.

Group LT2 Stakeholder Requirements Overview (Initial vision and scope text document provided by RJ, Felix, Jharyd, Renz, and Sim), February 3, 2026.

Group LT1 & LT2 Initial Requirements Elicitation Meeting [Video Recording]. YouTube. Available at: https://youtu.be/Sr2yzIQ4gug

Group LT1 & LT2 Scope Negotiation and MVP Finalization Meeting [Video Recording]. YouTube. Available at: https://youtu.be/nHiaiTB2a7w

Object Management Group (OMG). (2017). Unified Modelling Language (UML) Specification Version 2.5.1. Available at: https://www.omg.org/spec/UML/2.5.1/PDF

1.5 Definitions and Acronyms

ACID: Atomicity, Consistency, Isolation, Durability (Database transaction properties)

API: Application Programming Interface

BaaS: Backend-as-a-Service

COMET: Concurrent Object Modelling and Architectural Design Method

CRM: Customer Relationship Management

HTTPS/TLS: Hypertext Transfer Protocol Secure / Transport Layer Security

IEEE: Institute of Electrical and Electronics Engineers

MVP: Minimum Viable Product

PCI-DSS: Payment Card Industry Data Security Standard

PII: Personally Identifiable Information

RBAC: Role-Based Access Control

SMTP: Simple Mail Transfer Protocol

SRS: Software Requirements Specification

UML: Unified Modelling Language

2.0 SYSTEM OVERVIEW

The **Gordon Ramsay Restaurant Reservation System** is a web-based, single-tenant platform designed to replace manual logbooks with a real-time digital environment for a dedicated restaurant location. It serves as a centralized solution to optimize daily operations and provide a frictionless booking experience for diners.

Core Functionalities

- **Customer Portal**
  Enables users to search for real-time table availability, book reservations with a simulated deposit, join virtual waitlists, and manage personal profiles and dietary restrictions.

- **Restaurant Management**
  Provides staff with a dashboard featuring a **static, interactive floor plan grid** for real-time status updates (e.g., *Available*, *Occupied*, *Dirty*) and a **Guest CRM** to track customer history and VIP status.

- **Automated Communication**
  Dispatches booking confirmations, reminders, and waitlist alerts exclusively via **SMTP Email**.

Design & Context

- **Architecture**
  Built on a Client-Server model using Supabase as a Backend-as-a-Service (BaaS) for secure authentication, real-time data synchronization, and ACID-compliant PostgreSQL storage.

- **Methodology**
  Developed using the **COMET** (Concurrent Object Modelling and Architectural Design Method) and MVC (Model-View-Controller) patterns to ensure high modularity and concurrency control.

- **Key Constraints**
  The system prioritizes a 3-second UI load time and utilizes row-level locking to prevent double-booking during the checkout process.

3.0 SYSTEM ARCHITECTURE

3.1 Architectural Design

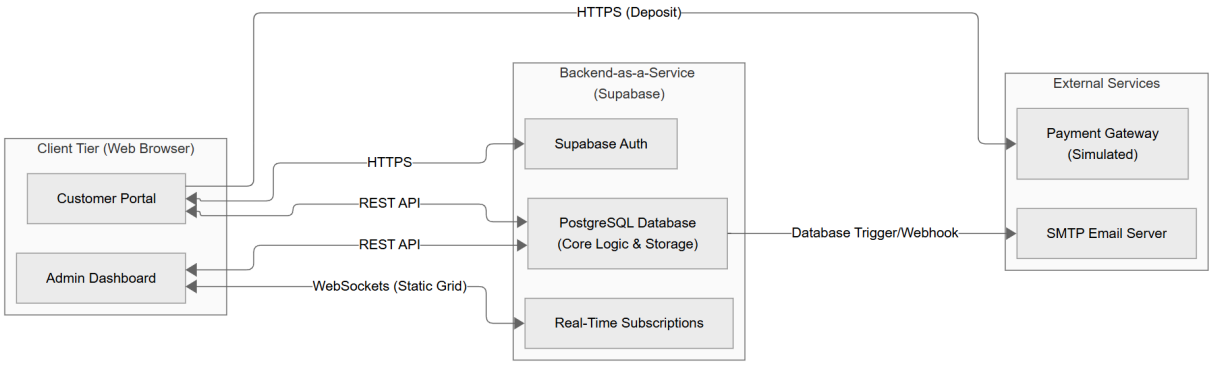
The Gordon Ramsay Restaurant Reservation System follows a **Three-Tier Client-BaaS**

(Backend-as-a-Service) **Architecture**. To achieve high maintainability and adhere to the single-tenant MVP constraints, the system's responsibilities are partitioned into the following high-level tiers:

- **Client/Presentation Tier (Frontend)**: A responsive, JavaScript-based web application that serves both the Customer Portal and the Admin Dashboard. It is responsible for rendering the UI, capturing the user inputs, managing client-side routing, and displaying the real-time interactive floor plan grid.

- **Data & Logic Tier (Supabase BaaS)**: Instead of a traditional custom server, the system relies on Supabase. This tier handles secure user authentication (Supabase Auth), relational data storage (PostgreSQL), and real-time database subscriptions (WebSockets). It enforces row-level security (RLS) and handles the 1-second concurrency locking required to prevent double-booking.

- **External Services Tier (Third-Party APIs)**: External Integrations required to fulfill specific functional capabilities, specifically the Payment Gateway for simulated deposit checkouts and the SMTP Email Server for automated notification dispatch.



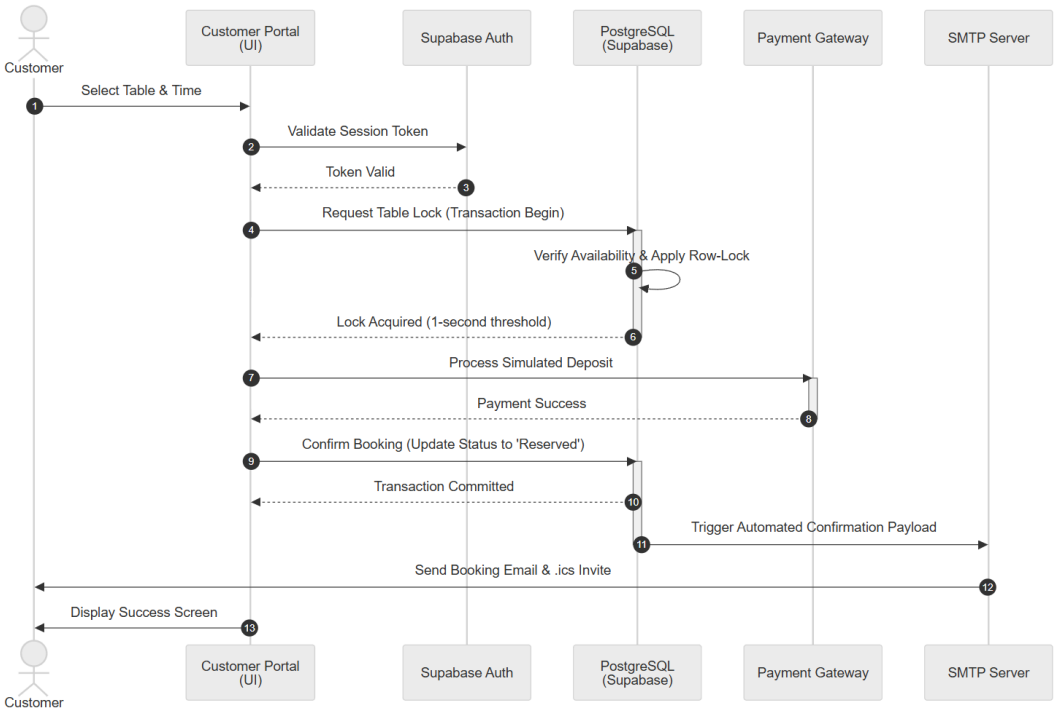
3.2 Decomposition Description

To handle the core operations of the reservation system, the architecture is broken down into the following functional subsystems:

- **Booking Engine Subsystem**:
  Responsible for querying table availability, applying combination logic for large party sizes, and executing the critical transaction flow. It interacts directly with the database's concurrency controller to secure a row-lock on a table during the simulated checkout phase.

- **Visual Table Management Subsystem**:
  Drives the static, interactive floor plan grid on the Admin Dashboard. It establishes a WebSocket connection to Supabase's Real-Time Subscriptions, ensuring that when a table's state changes in the database (e.g., from "Available" to "Occupied"), the grid's color-coding updates instantly across all connected admin devices without a page refresh.

- **Notification & Waitlist Subsystem**:
  A background-driven subsystem utilizing PostgreSQL database triggers. When a reservation state changes to "Cancelled," this subsystem automatically queries the Waitlist Queue, updates the next user's status, and constructs a JSON payload containing *.ics* calendar data, which is then dispatched to the external SMTP Email Server.



3.3 Design Rationale

The architecture described above was selected through careful consideration of the MVP

constraints, stakeholder negotiations, and technical trade-offs:

- **Selection of BaaS (Supabase) over Custom Backend:** Building a custom backend (e.g., Node.js/Express or Python/Django) would require significant developer overhead for socket management and authentication. Supabase was chosen because it provides out-of-the-box WebSocket synchronization (critical for the real-time static grid) and ACID-compliant PostgreSQL row-locking (critical for preventing double-bookings), allowing the team to meet the strict development timeline.

- **Single-Tenant vs. Multi-Tenant:** The initial stakeholder wishlist included a Super Admin dashboard for managing multiple restaurant franchises. This was scaled down to a Single-Tenant architecture to eliminate complex cross-database routing and role hierarchies, focusing strictly on a single Gordon Ramsay location to ensure system stability and MVP viability.

- **SMTP Notifications vs. SMS Push:** Premium SMS push notifications were traded for a standard SMTP Email architecture. This decision drastically reduces the recurring operational costs for the restaurant while still effectively mitigating double-bookings and fulfilling the waitlist automation logic.

- **Simulated Payment Gateway:** To maintain compliance with the Data Privacy Act of 2012 and avoid the severe legal liabilities associated with PCI-DSS data handling, the system architecture intentionally excludes the storage of Primary Account Numbers (PAN). The payment component strictly interfaces with a tokenized, simulated checkout environment.

4.0 DATA DESIGN

4.1 Data Description

The Gordon Ramsay Restaurant Reservation System transforms its information domain into structured data entities stored in a centralized Supabase PostgreSQL database. The major entities and their organization are as follows:

- **Reservations Table** – Stores booking details such as Reservation_ID, Date, Time, Party Size, Deposit Status, and Cancellation State.
- **Customers Table** – Stores user accounts, including Customer_ID, Name, Email, Contact Info, and Dietary Preferences.
- **Tables Entity** – Represents physical tables in the restaurant, linked to reservation status (Available, Reserved, Occupied, Dirty).
- **Waitlist Queue** – Stores customers waiting for cancellations, ordered by Waitlist_Position.
- **CRM Records** – Tracks guest history, allergies, VIP status, and no-show counts.
- **Menu Entity** – Stores menu items uploaded by administrators, including Item_ID, Name, Description, and Price.

All entities are synchronized in real time via Supabase's backend services to prevent double-booking and ensure operational integrity.

4.2 Data Dictionary

The following is an alphabetical list of the major system entities and attributes defined in

the Gordon Ramsay Restaurant Reservation System. Each entry specifies the data type and a concise description of its purpose within the system:

- **Contact_Info (String)** – Phone number or other contact details of the customer
- **CRM_Notes (Text)** – Guest history, allergies, VIP status, and personalized service notes
- **Customer_ID (Integer)** – Unique identifier for each customer
- **Date (Date)** – Reservation date
- **Deposit_Amount (Decimal)** – Required deposit for booking
- **Dietary_Preferences (String)** – Notes on allergies or food restrictions
- **Email (String)** – Used for login and notifications
- **Item_Description (Text)** – Description of the dish
- **Item_Name (String)** – Name of the dish
- **Item_Price (Decimal)** – Price of the dish
- **Menu_Item_ID (Integer)** – Unique identifier for menu items
- **Name (String)** – Full name of the customer
- **Party_Size (Integer)** – Number of guests in the reservation
- **Reservation_ID (Integer)** – Unique identifier for each booking
- **Reservation_Status (Enum: Reserved, Cancelled, Completed)** – Current state of the reservation
- **Table_ID (Integer)** – Unique identifier for each restaurant table
- **Table_Status (Enum: Available, Reserved, Occupied, Dirty)** – Current state of a table
- **Time (Time)** – Reservation time
- **Waitlist_Position (Integer)** – Queue order for customers waiting

5.0 COMPONENT DESIGN

This section provides a systematic breakdown of each subsystem introduced in Section 3.2. Each component is described using procedural description language (PDL) or pseudocode, with

references to local data structures where necessary.

5.1 Booking Engine Subsystem

Purpose: Implements reservation search, table-locking, deposit handling, and cancellation logic.
Traceability: FR-2, FR-3, FR-4, FR-10

```SQL
FUNCTION BookTable(date, time, partySize):
    availableTables ← Query Supabase(date, time)

    IF availableTables = ∅ THEN
        RETURN "No Availability" → Trigger Waitlist (FR-5)
    ENDIF

    IF partySize > MaxCapacity THEN
        combinedTables ← FindCombination(availableTables, partySize)
        IF combinedTables = ∅ OR partySize > 12 THEN
            RETURN "No Combination Found"
        ENDIF
    ENDIF

    LockTable(selectedTable, timeout=5min)   // Row-lock in PostgreSQL
    depositStatus ← SimulateCheckout(partySize, depositAmount)

    IF depositStatus = SUCCESS THEN
        CommitReservation(selectedTable, customerID)
        SendConfirmationEmail(customerID, reservationDetails)
    ELSE
        UnlockTable(selectedTable)
        RETURN "Payment Failed"
    ENDIF
END FUNCTION

FUNCTION CancelReservation(reservationID):
    Supabase.Update(reservationID.status = "Cancelled")
    TriggerWaitlistNotification(reservationID.timeSlot)
END FUNCTION
```

Local Data:

- Reservation(date, time, tableID, customerID, status)
- Deposit(amount, transactionID, status)

5.2 Visual Table Management Subsystem

Purpose: Provides administrators with a static, interactive floor plan grid with enforced colour coding.
Traceability: FR-7, FR-8

```SQL
FUNCTION UpdateTableStatus(tableID, newStatus):
    Authenticate(AdminSession)
    currentGrid ← FetchFloorPlan()
    DisplayGrid(currentGrid)

    IF tableID EXISTS THEN
        Supabase.Update(tableID.status = newStatus)
        ColorCode(newStatus) // Green, Yellow, Red, Grey
        BroadcastUpdate(tableID, newStatus) // via WebSocket
    ELSE
        RETURN "Invalid Table"
    ENDIF
END FUNCTION
```

Local Data:

- Table(tableID, capacity, status, color code)
- FloorPlan(gridLayout, tableStates)

5.3 Notification & Waitlist Subsystem

Purpose: Automates cancellation handling, waitlist promotion, and manual override.
Traceability: FR-5, FR-6, FR-10, FR-12

```SQL
TRIGGER OnReservationCancelled(reservationID):
    UpdateTableStatus(reservationID.tableID, "Available")
    nextCustomer ← Waitlist.PopNext()
    IF nextCustomer ≠ NULL THEN
        SendWaitlistOffer(nextCustomer, reservationDetails)
        StartTimer(10min)
        IF nextCustomer.AcceptedWithinWindow THEN
            ConfirmReservation(nextCustomer)
        ELSE
            nextCustomer.status ← "Revoked"
            TriggerWaitlistNotification(reservationID.timeSlot)
        ENDIF
    ENDIF
END TRIGGER

FUNCTION ManualWaitlistControl(adminAction, customerID):
    SWITCH(adminAction):
        CASE "Prioritize": MoveCustomerToFront(customerID)
        CASE "Remove": DeleteCustomerFromQueue(customerID)
        CASE "Insert": AddCustomerToQueue(customerID, position)
    END SWITCH
END FUNCTION
```

Local Data:

- Waitlist(queueID, customerID, timestamp, status)
- Notification(email, payloadJSON, status)

5.4 Guest CRM & Menu Management (Admin Extensions)

Purpose: Tracks customer history, manages menu items, and supports account deletion for compliance.
Traceability: FR-9, FR-11, LEG-1

```SQL
FUNCTION ViewGuestProfile(customerID):
    profile ← Supabase.Query(CRM where customerID)
    Display(profile)

FUNCTION ManageMenu(action, itemDetails):
    SWITCH(action):
        CASE "Add": Supabase.Insert(MenuItem)
        CASE "Edit": Supabase.Update(MenuItem)
        CASE "Delete": Supabase.Remove(MenuItem)
    END SWITCH
END FUNCTION

FUNCTION DeleteAccount(customerID):
    Supabase.Delete(UserAccount where customerID)
    Supabase.Delete(Reservations where customerID)
    Supabase.Delete(CRM where customerID)
    RETURN "Account Deleted - Right to Erasure"
END FUNCTION
```

Local Data:

- CRM(customerID, allergies, VIPstatus, history)
- MenuItem(itemID, name, description, price)
- UserAccount(customerID, credentials, PII)

6.0 HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

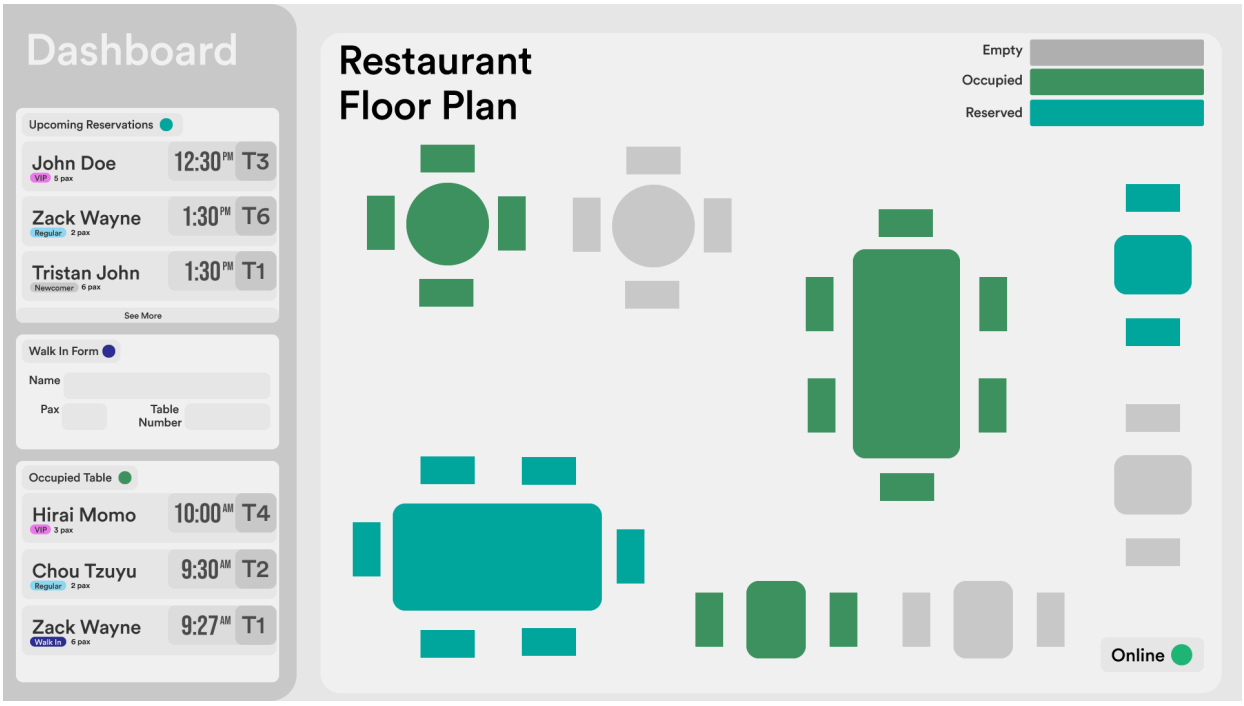
The system is a responsive, web-based, single-tenant platform comprising two modules: the Customer Portal and the Admin Dashboard.

- **Customer Portal (Diners):** The interface guides customers through a four-step reservation process (Search, Select Time, Details, Deposit). Users can securely log in via Supabase Auth to manage their profile and existing bookings, including self-cancellation. The system searches real-time table availability, manages table combination logic for large parties, and prompts users to join a digital waitlist when necessary.

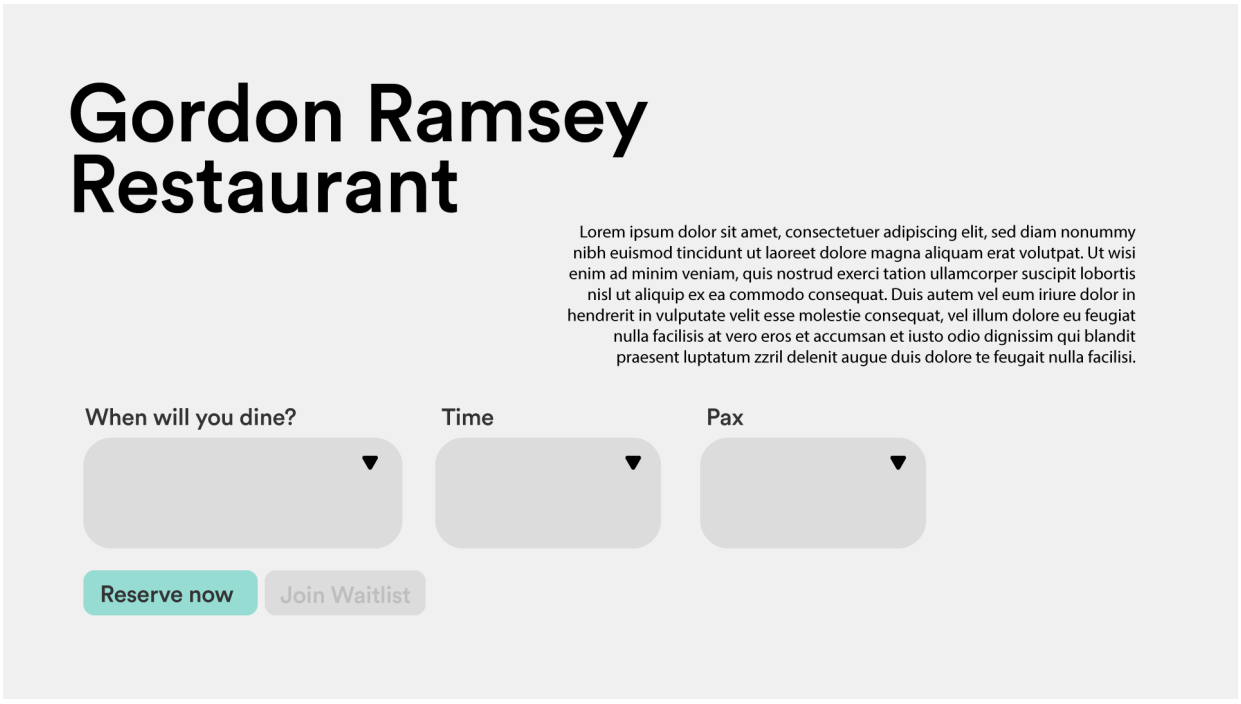
- **Admin Dashboard (Staff):** This module provides secure operational control. Staff use a static, interactive floor plan grid with real-time color-coding to update table statuses (Available, Occupied, Reserved). Administrators can also manage all bookings (list/calendar view), manually enter walk-ins, block dates, and access detailed Guest CRM records (history, VIP status, no-shows, and allergies).

- **System Feedback:** Real-time table status updates are instantly reflected on the Admin grid via color-coding. Customers receive automated booking confirmations and calendar invites via SMTP Email upon successful simulated deposit checkout. The Admin Dashboard displays real-time connection status for critical external services (Supabase, Payment Gateway, SMTP) and an "Offline Warning" during data desynchronization.

6.2 Screen Images



Admin User Interface



Customer User Interface

6.3 Screen Objects and Actions

- **Date, Time, Party Size Inputs (Customer)**: Enters parameters to query the Supabase database for real-time table availability (FR-2).
- **"Confirm Deposit" Button (Customer):** Initiates the secure simulated checkout process to lock the selected table and commit the reservation (FR-3, U1).
- **"Join Waitlist" Button (Customer):** Clicks to log a request in the Supabase backend for a fully booked time slot (FR-5, U3).
- **Table Icon on Floor Plan (Restaurant Admin):** Clicks to open a dropdown menu and manually update the Table_Status (e.g., from "Reserved" to "Occupied") (FR-7, U2).
- **Table Status Dropdown Menu (Restaurant Admin):** Selects a new status, which updates the color-coding of the table on the grid in real-time (FR-7, U2).
- **"Add Walk-in" Button (Restaurant Admin):** Clicks to manually input a customer's name and party size, assigning them to an available table (FR-8, U5).
- **Customer Name/Profile Link (Restaurant Admin):** Clicks to display the guest's CRM profile, including past dining history, allergy notes, and VIP status (FR-9, U6).
- **Menu Management Forms (Restaurant Admin):** Uses inputs to upload, edit, or remove menu items displayed on the Customer Portal (FR-11).
- **Waitlist Priority Controls (Restaurant Admin):** Uses controls to manually prioritize, remove, or insert customers into the Waitlist Queue (FR-12).

7.0 REQUIREMENTS MATRIX

Requirements ID	Requirements Description	Component
FR-1	The system shall allow users to register, securely log in, and manage their profile details (including contact information and dietary restrictions) via the Supabase authentication backend.	• Customer Portal • Supabase Auth
FR-2	The system shall allow customers to search for real-time table availability by specifying a Date, Time, and Party Size (Pax), while also providing access to a view-only digital menu before booking	• Customer Portal • PostgreSQL Database
FR-3	The system shall securely lock an available table upon user selection and utilize a simulated checkout process to handle required reservation deposits	• PostgreSQL Database • Payment Gateway
FR-4	The system shall automatically calculate and combine adjacent small tables (e.g., tow 4-seaters) if a requested Party Size exceeds standard individual table capacities	• PostgreSQL Database
FR-5	The system shall provide a digital waitlist for fully booked time slots and automatically notify the next queued customer if a cancellation occurs.	• Customer Portal • PostgreSQL Database • SMTP Email Server
FR-6	The system shall automatically dispatch booking confirmations, calendar invites (.ics), and upcoming reminders exclusively via SMTP Email Integration.	• PostgreSQL Database • SMTP Email Server
FR-7	The system shall provide administrators with a static, interactive floor plan grid reflecting the restaurant's actual layout, allowing staff to manually click and update real-time table statuses (e.g,. Available, Occupied, Reserved, Dirty)	• Admin Dashboard • PostgreSQL Database • Real-Time Subscriptions
FR-8	The system shall allow administrators to view bookings in both list and calendar formats, manually enter phone/walk-in reservations, and block out dates for holidays or private events.	• Admin Dashboard • PostgreSQL Database • Real-Time Subscriptions
FR-9	The system shall track and display individual customer metrics to the restaurant staff, including past dining history, VIP status, no-show counts, and specific allergy notes.	• Admin Dashboard • PostgreSQL Database • Real-Time Subscriptions
FR-10	The system shall allow customers to cancel existing, upcoming reservations through their account dashboard. Upon cancellation, the system shall automatically update the table’s status in the Supabase database to “Available” and trigger the waitlist notification protocol.	• Customer Portal • PostgreSQL Database
FR-11	The system shall allow administrators to upload, edit, or remove items from the digital menu displayed on the Customer Portal.	• Admin Dashboard • PostgreSQL Database • Real-Time Subscriptions

FR-12	The system shall allow Restaurant Administrators to manually view, prioritize, and edit customers on the Waitlist Queue to accommodate VIPs or walk-in emergencies.	• Admin Dashboard • PostgreSQL Database • Real-Time Subscriptions
FR-13	The Admin Dashboard shall display real-time connection status indicators for the system's three critical external dependencies (Supabase, Payment Gateway, and SMTP Email Server).	• Admin Dashboard • Supabase Auth • Real-Time Subscriptions • Payment Gateway • SMTP Email Server
PR-1	The web application shall load the customer-facing real-time availability grid and the administrator's static floor plan within 3 seconds over a standard 4G/LTE or broadband	• Customer Portal • PostgreSQL Database
PR-2	The system shall utilize the Supabase's real-time database row-locking mechanisms to resolve conflicts within 1 second if multiple customers attempt to reserve the exact same table at the exact same time	• PostgreSQL Database
PR-3	Automated booking confirmations shall be dispatched to the SMTP server within 10 seconds of a successful simulated deposit checkout	• PostgreSQL Database • Payment Gateway • SMTP Email Server
SAF-1	The system shall early on Supabase's automated Point-in-Time Recovery (PITR) and daily backups to prevent the catastrophic loss of guest CRM data and future reservations in the event of a database failure	• PostgreSQL Database • Real-Time Subscriptions
SAF-2	If the restaurant's operational dashboard loses its internet connection, the system must clearly display an "Offline Warning" to the staff. It must disable the ability to change table status until the connection is restored to prevent staff from making operational decisions based on desynchronized data	• Admin Dashboard • Real-Time Subscriptions
SEC-1	All system access shall require secure authentication managed via Supabase Auth. The system must strictly separate Customer accounts from Restaurant Administrator accounts using Role-Based Access Control (RBAC)	• Customer Portal • Admin Dashboard • Supabase Auth
SEC-2	To protect mobile and desktop connections, all web traffic between the client browsers and Supabase backend shall be strictly encrypted using standard HTTPS/TLS protocols	• PostgreSQL Database
SEC-3	To mimic PCI-DSS compliance for the MVP, the system shall not store raw credit card numbers in the local database. All deposit transactions must be handled via a secure, tokenized simulated checkout gateway	• Payment Gateway

DB-1	The system shall utilize a PostgreSQL relational database (hosted via Supabase) to ensure ACID (Atomicity, Consistency, Isolation, Durability) compliance. This guarantees that if a reservation is interrupted midway, the database will not save a partial or corrupted booking.	• PostgreSQL Database • Real-Time Subscriptions
DB-2	Customer profiles and Guest CRM data shall be retained securely in the database indefinitely, unless a customer explicitly requests the deletion of their account	• Customer Portal • Supabase Auth
DB-3	The database shall store all reservation date and time data strictly in UTC standard format, and automatically convert it to the restaurant's local time zone for display on the Customer Portal and Admin Dashboard.	• Customer Portal • Admin Dashboard • Real-Time Subscriptions
LEG-1	The system shall comply with Republic Act No. 10173 (Data Privacy Act of 2012) regarding the collection and storage of Personally Identifiable Information (PII). Users must be presented with a mandatory consent checkbox regarding the storage of their email addresses and dietary restrictions during account registration.	• Customer Portal • Supabase Auth • SMTP Email Server
LEG-2	To mitigate legal liability for the LT1 developer team during the MVP phase, the system is strictly prohibited from capturing, storing, or transmitting real credit card Primary Account Numbers (PAN). The deposit system must remain a simulated checkout environment. If the software is ever upgraded to a production release, it must integrate a fully PCI-DSS-complaint third-party payment gateway (e.g., Stripe or PayMongo).	• Customer Portal • Supabase Auth • Payment Gateway